# Analyse d'identité d'auteurs et détection de plagiat

CHEDJARI EL MEUR Hadjer
NANMEDIGOU Yendoumban
Université de Caen Normandie

Mars 2025

**Résumé**

Ce rapport présente la conception et l'implémentation d'un système de détection de plagiat utilisant des techniques de tokenisation, de vectorisation (CountVectorizer, TF-IDF) et divers classificateurs (Naïve Bayes, Perceptron, etc.).

## Table des matières

**1 Introduction** **3**

**2 Objectifs du Projet** **3**
2.1 Prétraitement et Extraction des Caractéristiques . . . . . 3
2.2 Développement des Modèles de Classification . . . . . 3
2.3 Optimisation et Expérimentations . . . . . 4

**3 Implémentation et Algorithmes** **5**
3.1 Tokenization . . . . . 5
3.2 Extraction des caractéristiques . . . . . 5
3.2.1 CountVectorizer . . . . . 5
3.2.2 TF-IDF . . . . . 6
3.3 Modéles de classification . . . . . 7
3.3.1 Naïve Bayes . . . . . 7
3.3.2 Logistic Regression . . . . . 7
3.3.3 Perceptron . . . . . 9
3.3.4 Decision Tree . . . . . 10
3.3.5 Bagging (Bootstrap Aggregating) . . . . . 11
3.3.6 AdaBoost (Adaptive Boosting) . . . . . 11

**4 Architecture** **12**
4.1 Paquetage . . . . . 12
4.1.1 model . . . . . 13
4.1.2 experiments . . . . . 14
4.1.3 gui . . . . . 14
4.1.4 util . . . . . 14
4.1.5 tests . . . . . 14
4.2 Chaîne de traitement . . . . . 15

**5 Experimentations** **15**
5.1 Pré-analyse du corpus de données utilsé . . . . . 15
5.1.1 Description . . . . . 15
5.1.2 Tailles du corpus . . . . . 16
5.1.3 Qu'avons-nous analysé ? . . . . . 16
5.2 Évaluations de nos modèles . . . . . 17
5.2.1 Métriques évaluées . . . . . 17
5.2.2 Quels modèles sont les plus performants pour détecter du plagiat ? . 18
5.3 Interface graphique . . . . . 21
5.4 Limites . . . . . 22

**6 Conclusion** **22**

# 1 Introduction

Avec l'augmentation constante du volume de documents mis en ligne, le plagiat est devenu une problématique centrale dans de nombreux domaines, notamment en milieu académique et juridique. Identifier l'origine d'un document ou d'une partie de texte est un défi important pour assurer l'intégrité des contenus et lutter contre la fraude intellectuelle.

L'objectif de ce projet est de mettre en œuvre des techniques de traitement automatique du langage naturel (*NLP*) afin d'analyser le contenu textuel et d'évaluer la similarité entre documents. Cette approche repose sur plusieurs étapes clés : la transformation des textes en représentations exploitables, l'implémentation de métriques de comparaison, et l'utilisation de modèles d'apprentissage automatique pour classer et regrouper les textes selon leur origine.

Dans ce cadre, nous avons choisi de développer notre propre pipeline de détection, sans recourir à des implémentations toutes faites. Cela inclut la création d'un tokenizer, la vectorisation des documents, ainsi que l'implémentation et l'évaluation de différents algorithmes de classification. Une analyse quantitative des résultats sera menée afin de comparer les performances des différentes approches et d'identifier les facteurs influençant leur efficacité.

Ce rapport détaille les choix méthodologiques adoptés, les techniques mises en œuvre et les résultats obtenus, dans le but de mieux comprendre les enjeux liés à la détection de plagiat et d'évaluer l'efficacité des différentes stratégies d'analyse.

# 2 Objectifs du Projet

Pour concevoir un système de détection de plagiat, nous avons défini plusieurs objectifs techniques et méthodologiques :

## 2.1 Prétraitement et Extraction des Caractéristiques

L'une des étapes clés du projet consiste à transformer un texte brut en une représentation exploitable par un algorithme de classification. Pour cela, nous avons :

- Développé un tokenizer capable de gérer la ponctuation, la casse, les stopwords et la lemmatisation.
- Expérimenté différentes stratégies d'**n-grams** afin d'améliorer la capture des structures linguistiques.
- Implémenté nos propres versions de **Bag-of-Words** et **TF-IDF**.

## 2.2 Développement des Modèles de Classification

Nous avons choisi d'implémenter plusieurs modèles afin de comparer leurs performances et d'identifier ceux les plus adaptés à notre problème. Nos implémentations incluent :

- **Naïve Bayes**, pour sa rapidité et son efficacité sur des données textuelles.
- **Perceptron et Régression Logistique**, pour tester des approches linéaires d'apprentissage.
- **Arbre de Décision, Random Forest et AdaBoost**, afin d'évaluer des approches non linéaires basées sur des ensembles de modèles.

## 2.3 Optimisation et Expérimentations

Une attention particulière a été portée à l'évaluation et à l'optimisation des modèles :

- Nous avons mis en place un **early stopping** pour éviter le sur-apprentissage et optimiser le temps d'entraînement.
- Une évaluation détaillée des performances a été réalisée en utilisant les métriques **précision, rappel et F1-score**.
- Nous avons analysé l'impact des hyperparamètres (taille des n-grams, régularisation, profondeur des arbres) sur les performances.

En suivant cette méthodologie, notre objectif est de construire une solution fiable et adaptable, tout en garantissant une compréhension approfondie des algorithmes sous-jacents.

# 3 Implémentation et Algorithmes

## 3.1 Tokenization

La première étape du pipeline consiste à découper le texte en tokens. Nous avons appliqué :

**Algorithm 1** Tokenization

1: **Entrée** : Texte brut $T$
2: **Sortie** : Liste de tokens $L$
3: Convertir $T$ en minuscules
4: Supprimer la ponctuation
5: Découper $T$ en mots (tokens)
6: Supprimer les stopwords de $T$
7: Appliquer la lemmatisation sur les tokens
8: **Retourner** $L$

**Complexité :** La complexité de la tokenisation dépend du type d'approche utilisée :

— Pour une tokenization basée sur des expressions régulières :

$$O(n) \tag{1}$$

où $n$ est la longueur du texte.

— Pour une tokenization basée sur un dictionnaire avec recherche binaire :

$$O(n \log k) \tag{2}$$

où $k$ est la taille du dictionnaire.

## 3.2 Extraction des caractéristiques

Cette étape transforme les tokens générés précédemment en représentations numériques exploitables par les algorithmes d'apprentissage automatique.

Nous avons utilisé deux méthodes :

— **CountVectorizer** : Nombre d'occurrences de chaque mot

— **TF-IDF** : Importance relative des mots

Pour cela on distingue 2 parties principales :

- Indexation : Associe un index unique à chaque mot du vocabulaire.
- Représentation : Génère une matrice où :
  Chaque ligne représente un document.
  Chaque colonne correspond à un mot du vocabulaire.

### 3.2.1 CountVectorizer

CountVectorizer transforme un texte en une matrice de fréquences des mots.

**Algorithm 2** CountVectorizer

1: **Entrée** : Liste de documents de test $D = \{d_1, d_2, \ldots, d_N\}$
2: **Sortie** : Matrice de fréquence des termes $M$ **chaque document** $d_i$ **chaque mot** $w_j$ dans $d_i$
3: $M[i, j] \leftarrow$ nombre de fois que $w_j$ apparait dans $d_i$
4: **Retourner** la matrice $M$

**Complexité :** La transformation d'un corpus contenant $n$ documents et $m$ mots uniques dans le vocabulaire est :

$$O(n \cdot m) \tag{3}$$

où :

— $n$ : nombre de documents.
— $m$ : nombre total de mots uniques.

### 3.2.2 TF-IDF

Cette d'extraction de caractéristiques textuelles Pondère les mots en fonction de :
- Leur fréquence dans un document.
- Leur rareté dans l'ensemble du corpus.

L'équation de TF-IDF est :

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{DF}(t)}$$

où :

— $TF(t, d)$ : fréquence du terme $t$ dans le document $d$.
— $f_t$ : nombre d'occurrences du terme $t$ dans $d$.
— $N$ : nombre total de mots dans $d$.
— $IDF(t)$ : inverse de la fréquence du document.
— $N_d$ : nombre total de documents.
— $N_t$ : nombre de documents contenant le terme $t$.

**Algorithm 3** Calcul de TF-IDF

1: **Entrée** : Liste de documents $D = \{d_1, d_2, \ldots, d_N\}$
2: **Sortie** : Matrice TF-IDF $M$ **chaque document** $d_i$ **chaque terme** $t_j$ dans $d_i$
3: $\text{TF}(t_j, d_i) \leftarrow \frac{\text{nombre de fois que } t_j \text{ apparait dans } d_i}{\text{total des mots dans } d_i}$
4: $\text{DF}(t_j) \leftarrow$ nombre de documents contenant $t_j$
5: $\text{IDF}(t_j) \leftarrow \log\left(\frac{N}{\text{DF}(t_j)}\right)$
6: $M[i, j] \leftarrow \text{TF}(t_j, d_i) \times \text{IDF}(t_j)$
7: **Retourner** la matrice TF-IDF $M$

**Complexité :** Le calcul du TF-IDF a une complexité de :

$$O(n \cdot m) \tag{4}$$

où :

— $n$ : nombre de documents.
— $m$ : nombre de mots uniques dans le corpus.

## 3.3 Modéles de classification

A ce niveau les matrices de caractéristiques sont Utilisées pour entraîner le classificateur et prédire si un document est un "plagiat" ou "non plagiat".

### 3.3.1 Naïve Bayes

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)} \tag{5}$$

où :

- $P(y|X)$ : probabilité a posteriori de la classe $y$ sachant les caractéristiques $X$.
- $P(X|y)$ : probabilité d'observer $X$ sachant que l'étiquette est $y$.
- $P(y)$ : probabilité a priori de la classe $y$.
- $P(X)$ : probabilité totale d'observer $X$.

**Algorithm 4** Naïve Bayes pour classification

1: **Entrée** : Ensemble de données d'entraînement $\{(x_i, y_i)\}$, où $x_i$ est un vecteur de caractéristiques et $y_i$ est la classe
2: **Sortie** : Modèle de classification **chaque** classe $y$
3: Calculer la probabilité a priori $\hat{P}(y) = \frac{\text{nombre de } y \text{ dans les données d'entraînement}}{N}$ **chaque** caractéristique $x_j$
4: Estimer la probabilité conditionnelle $\hat{P}(x_j|y)$ en utilisant les données d'entraînement
5: **Retourner** le modèle avec $\hat{P}(y)$ et $\hat{P}(x_j|y)$

**Complexité :** L'entraînement de Naïve Bayes consiste à calculer les probabilités conditionnelles pour chaque attribut et chaque classe.

$$O(n \cdot m) \tag{6}$$

où :

— $n$ : nombre d'échantillons dans l'ensemble d'entraînement.
— $m$ : nombre de caractéristiques (features).

La classification d'un nouvel échantillon se fait en :

$$O(m \cdot k) \tag{7}$$

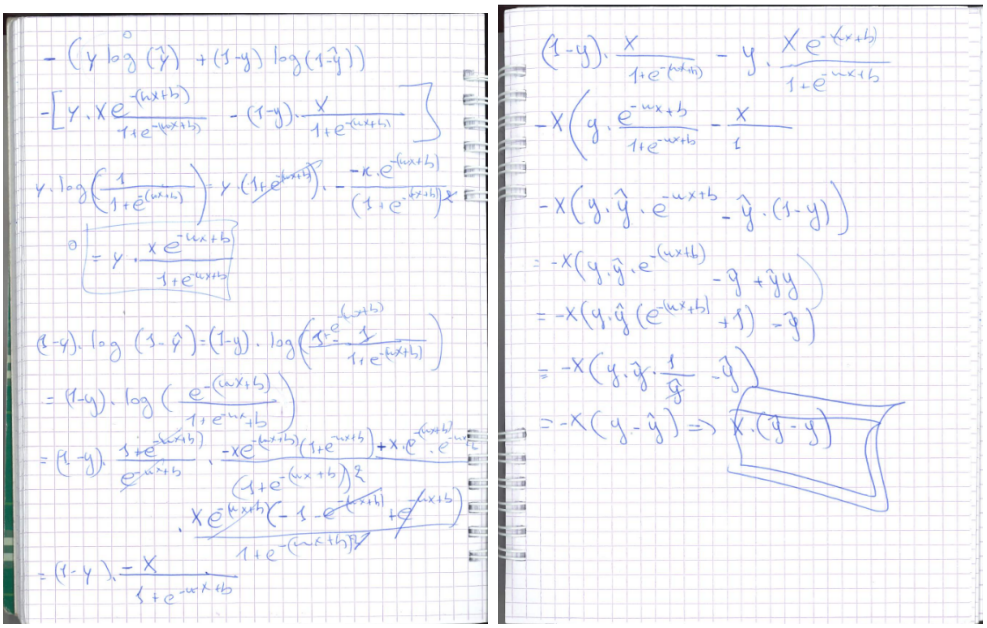
où $k$ est le nombre de classes.

### 3.3.2 Logistic Regression

$$P(y = 1|X) = \frac{1}{1 + e^{-(wX+b)}} \tag{8}$$

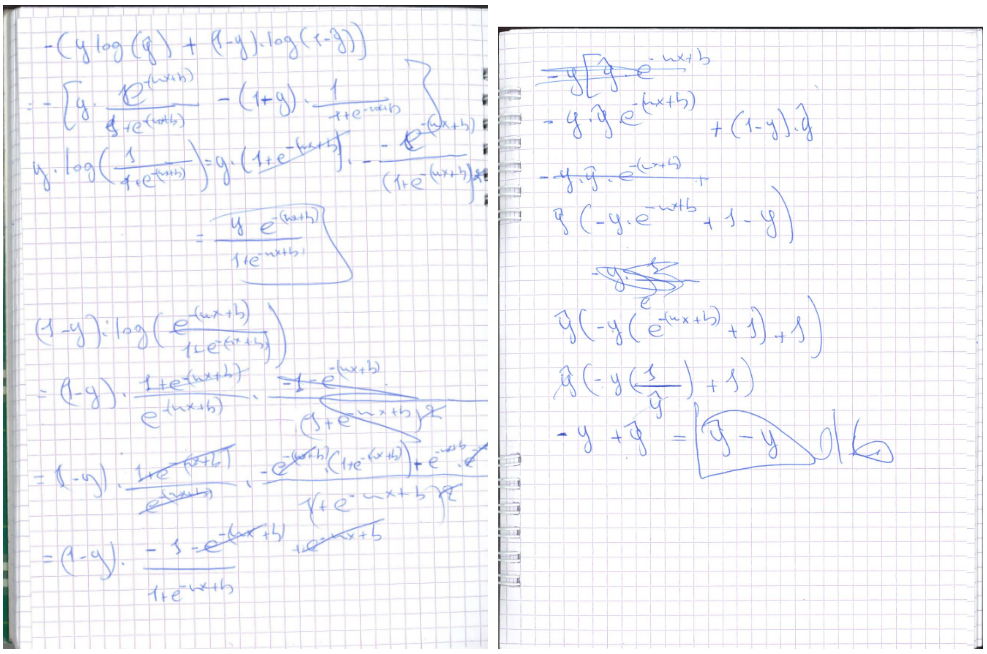
où :

- $P(y = 1|X)$ : probabilité que $X$ appartienne à la classe $y = 1$.
- $w$ : vecteur des poids associés aux caractéristiques.
- $X$ : vecteur des caractéristiques de l'instance.
- $b$ : biais du modèle.
- $e$ : base du logarithme naturel ($\approx 2.718$).



(a) Original Image (b) After Histogram Normalization

FIGURE 1 – Calcule de Dw



(a) Original Image (b) After Histogram Normalization

FIGURE 2 – Calcule de Db

**Algorithm 5** Régression Logistique pour classification

1: **Entrée** : Ensemble de données d'entraînement $\{(x_i, y_i)\}$, où $x_i$ est un vecteur de caractéristiques et $y_i$ est la classe
2: Initialiser les poids $w$ et le biais $b$ **chaque** échantillon $(x_i, y_i)$
3: Calculer la probabilité $\hat{y}_i = \sigma(w \cdot x_i + b)$, où $\sigma(z) = \frac{1}{1+e^{-z}}$
4: Calculer l'erreur $L = y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$
5: Mettre à jour les poids $w$ et $b$ avec la descente de gradient :
6: $w = w - \eta \frac{\partial L}{\partial w}$
7: $b = b - \eta \frac{\partial L}{\partial b}$
8: **Retourner** $w, b$

**Complexité :** L'entraînement est effectué via la descente de gradient, qui prend :

$$O(i \cdot n \cdot m) \tag{9}$$

où :

- $i$ : nombre d'itérations.
- $n$ : nombre d'échantillons.
- $m$ : nombre de caractéristiques.

La classification d'un nouvel échantillon se fait en :

$$O(m) \tag{10}$$

### 3.3.3 Perceptron

$$y = \text{sign}(wX + b) \tag{11}$$

où :

- $y$ : classe prédite ($+1$ ou $-1$).
- $\text{sign}(z)$ : fonction qui retourne $+1$ si $z$ est positif, sinon $-1$.
- $w$ : vecteur des poids.
- $X$ : vecteur des caractéristiques.
- $b$ : biais.

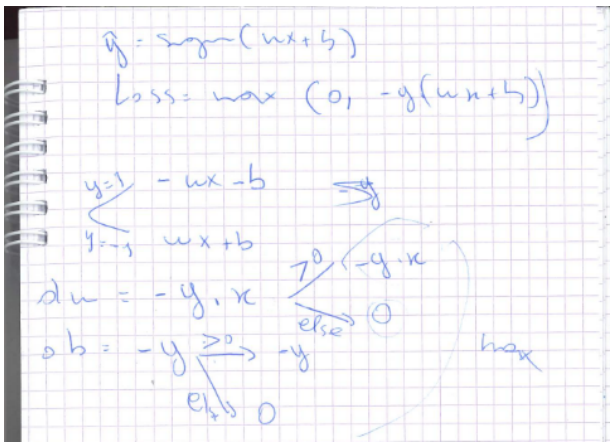



FIGURE 3 – Calcule de Decent Gradient

**Algorithm 6** Perceptron pour classification binaire

1: **Initialisation** : poids $w$, biais $b$, taux d'apprentissage $\eta$ **chaque** échantillon $(x_i, y_i)$
2: $\hat{y} = \text{sign}(w \cdot x_i + b)$ $\hat{y} \neq y_i$
3: $w = w + \eta(y_i x_i)$
4: $b = b + \eta y_i$
5: **Retourner** $w, b$

**Complexité :** L'algorithme du perceptron apprend en ajustant les poids en fonction des erreurs. Sa complexité d'entraînement est :

$$O(i \cdot n \cdot m) \tag{12}$$

et la classification d'un nouvel échantillon se fait en :

$$O(m) \tag{13}$$

### 3.3.4 Decision Tree

**Indice de Gini**

$$Gini = 1 - \sum p_i^2 \tag{14}$$

où :

- $Gini$ : mesure d'impureté du nœud.
- $p_i$ : proportion d'éléments appartenant à la classe $i$ dans le nœud.

**Entropie**

$$Entropy = -\sum p_i \log_2 p_i \tag{15}$$

où :

- $Entropy$ : mesure d'incertitude dans la répartition des classes.
- $p_i$ : proportion d'éléments de la classe $i$ dans le nœud.
- $\log_2$ : logarithme en base 2.

**Algorithm 7** Arbre de Décision pour classification

1: **Entrée** : Ensemble de données d'entraînement $\{(x_i, y_i)\}$, où $x_i$ est un vecteur de caractéristiques et $y_i$ est la classe
2: **Sortie** : Arbre de décision
3: Construire l'arbre récursivement : **chaque** noeud de l'arbre
4: Calculer l'impureté (par exemple, entropie ou Gini) pour chaque caractéristique
5: Choisir la caractéristique qui minimise l'impureté (meilleure séparation)
6: Diviser l'ensemble de données en fonction de la caractéristique choisie
7: Si le noeud est pur (impureté = 0) ou si un critère d'arrêt est atteint, arrêter
8: **Retourner** l'arbre construit

**Complexité :** L'entraînement d'un arbre de décision nécessite la recherche du meilleur attribut pour la division des données à chaque nœud. Cette opération a une complexité :

$$O(n \cdot m \log n) \tag{16}$$

La classification d'un nouvel échantillon prend :

$$O(d) \tag{17}$$

où $d$ est la profondeur de l'arbre.

### 3.3.5 Bagging (Bootstrap Aggregating)

Bagging est une méthode d'ensemble qui vise à réduire la variance des modèles d'apprentissage supervisé. Il repose sur la création de plusieurs sous-échantillons des données d'entraînement en effectuant un **bootstrap sampling** (tirage avec remise). Chaque modèle est entraîné indépendamment sur un sous-échantillon et leurs prédictions sont ensuite combinées pour obtenir un modèle plus robuste. Le RandomForest en est une extension (un Bagging sur des arbres de décision).

**Algorithm 8** Algorithme de Bagging

1: **Entrée** : Ensemble d'entraînement $D$ de taille $N$, Nombre de modèles $M$, Modèle de base $h$ $t = 1$ **to** $M$
2: Générer un échantillon $D_t$ en tirant $N$ instances de $D$ avec remise.
3: Entraîner un modèle $h_t$ sur $D_t$
4: **Prédiction** : Classification
5: Retourner la majorité des prédictions des $M$ modèles : majorité$(h_1(x), h_2(x), ..., h_M(x))$
6: Retourner la moyenne des prédictions des $M$ modèles : $\frac{1}{M}\sum_{t=1}^{M} h_t(x)$

**Algorithme de Bagging**

### 3.3.6 AdaBoost (Adaptive Boosting)

AdaBoost est une technique de **boosting** qui combine plusieurs classificateurs faibles pour créer un modèle robuste. Contrairement au bagging, AdaBoost entraîne les modèles **séquentiellement** en pondérant les erreurs : les exemples mal classés reçoivent un poids plus important à chaque itération pour forcer le modèle suivant à mieux les apprendre.

**Algorithm 9** Algorithme d'AdaBoost

1: **Entrée** : Ensemble d'entraînement $D = \{(x_1, y_1), ..., (x_N, y_N)\}$, Nombre de modèles $M$, Modèle faible $h$, Poids initialisés uniformément : $w_i = \frac{1}{N}$ $t = 1$ **to** $M$
2: Entraîner un classificateur $h_t$ sur les données pondérées $(D, w)$
3: Calculer l'erreur pondérée $e_t = \sum_{i=1}^{N} w_i \cdot \mathbb{1}(h_t(x_i) \neq y_i)$
4: Calculer le poids du modèle $\alpha_t = \frac{1}{2}\ln\left(\frac{1-e_t}{e_t}\right)$
5: Mettre à jour les poids des échantillons :

$$w_i \leftarrow w_i \cdot e^{-\alpha_t y_i h_t(x_i)}$$

6: Normaliser les poids $w_i$ pour que $\sum w_i = 1$
7: **Prédiction** :
8: Prédire la classe pour un nouvel exemple $x$ :
H(x) = sign $\left(\sum_{t=1}^{M} \alpha_t h_t(x)\right)$

**Algorithme d'AdaBoost**

# 4 Architecture

## 4.1 Paquetage

Répondre aux objecttifs de notre projet a nécessité d'organiser notre travail en plusieurs packages et sous packages, chacun ayant un rôle bien défini.

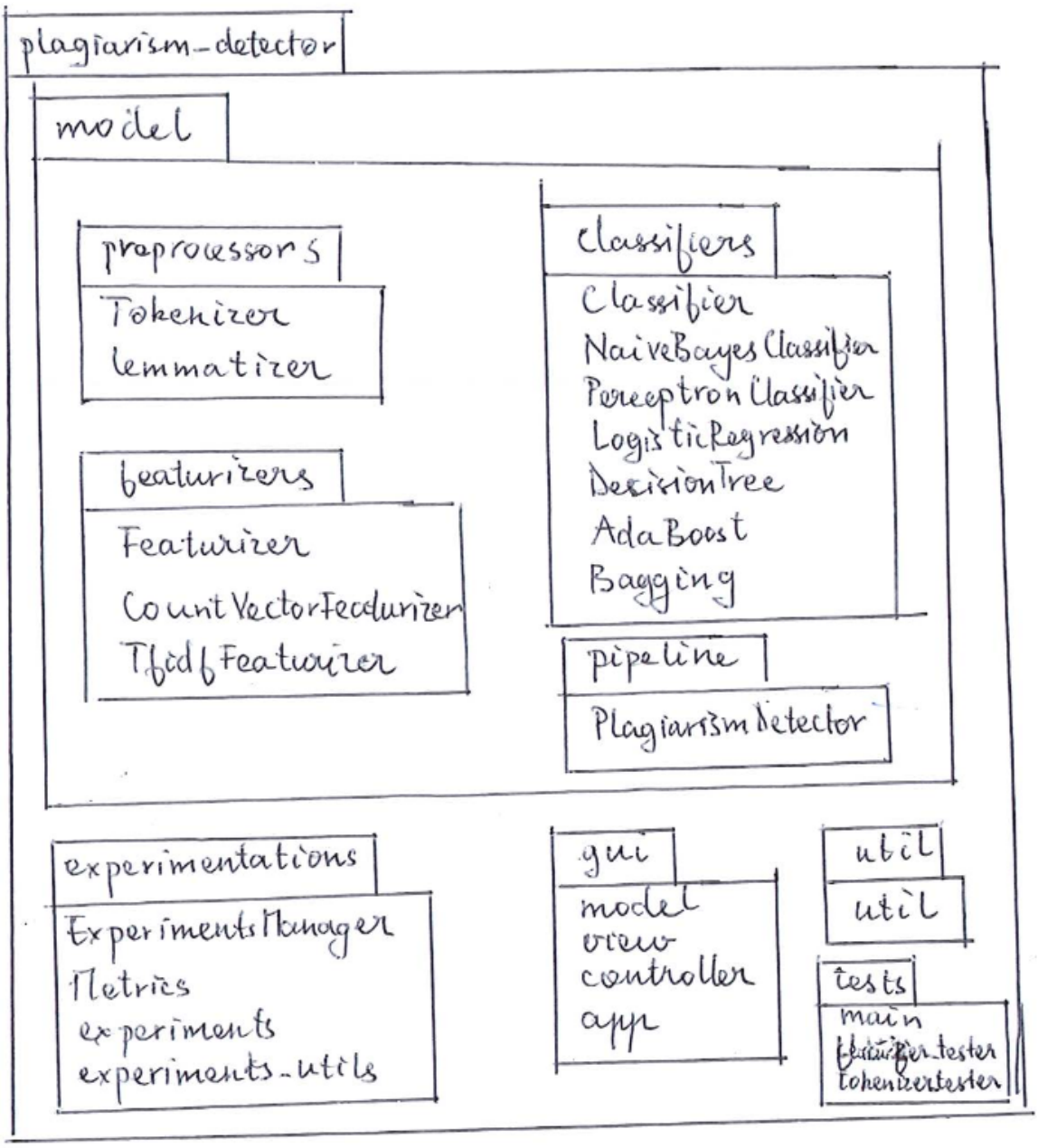



FIGURE 4 – Paquetage

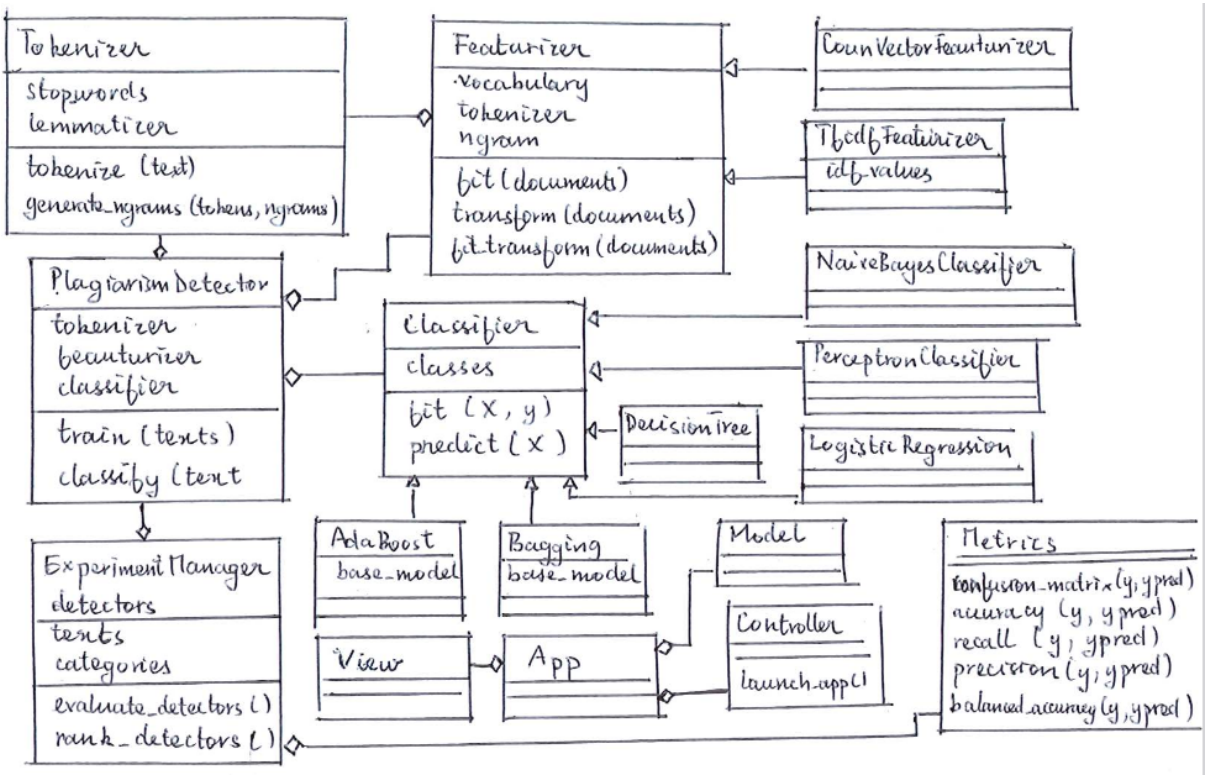



FIGURE 5 – Diagramme de classes

### 4.1.1 model

C'est le package central du fait qu'il contient toute la logique métier du détecteur de plagiat implémenté, des pré-processeurs de texte aux classifiers. Il est lui-même composé de 4 sous packages.

**preprocessors** Les outils nécessaire au pré-traitement de texte, notamment le Tokenizer et le lemmatizer, sont contenu dans ce package. Le Tokenizer a pour fonction la première étape de notre chaîne de traitement. À partir d'un texte brut, quel sont les élements (mots ou sac de mots) que nous jugeons pertinents pour une extraction de caractéristiques. Pour cela, il dispose de la méthode `tokenize` qui prend en entrée le texte à tokenizer. Son attribut `stop_words` (qui est initialisé par défaut aux stopwords anglais du module `nltk`), nous permettra de préciser quels mots ne sont pas nécessaires pour l'étape suivante. Quant à son `lemmatizer` (une fonction, initialisée à `None`), il nous permettra d'appliquer ou pas d'appliquer la lemmatization aux mots du texte traité. Notre lemmatizer est la fonction `lemmatize` du fichier `lemmatizer.py` qui prend en entrée une liste de mots et renvoie leur formes canoniques. Elle utilise pour cela le module `spacy`.

**featurizers** Après la tokenization, l'extraction des caractéristiques sur les tokens générés est assurée par les featurizers, contenus dans ce package. Leur modèle est établi par la classe abstraite `Featurizer`. Un featurizer stocke le vocabulaire qu'il a appris, pour le réutiliser lors de l'extraction de traits, et dispose d'un tokenizer pour s'assurer que les éléments traités soient des tokens ; d'où ses attributs `vocabulary` et `tokenizer`. `n_grams` est son attribut qui nous permettra de préciser les tailles des mots de sacs voulus. Quant aux méthodes, on peut lui faire apprendre du vocabulaire en lui donnant une liste de

textes (méthode `fit`) et lui demander d'extraire les caractéristiques correspondant à une liste de textes (méthode `transform`). Sa méthode `fit_transform` combine les deux étapes mentionnées précédemment. Les classes et `CountVectorFeaturizer` et `TfidfFeaturizer` sont les deux featurizers implémentés.

**classifiers** C'est la prochaine étape, classification des textes (en réalité des caractéristiques extraites), qu'assure cet package. Un classifier, dont le modèle est défini par la classe abstraite `Classifier` peut être entraîné sur un ensemble de textes. C'est le rôle de sa méthode `fit`. Pour cela, on lui fournit la matrice de features où chaque ligne représente les caractéristique d'un texte, puis les classes auxqueles appartiennent ces textes. On pourra ensuite lui demander de déterminer la classe d'un ensemble de textes (méthode `predict`) en lui fournissant les matrices de features. Il stocke par ailleurs dans un attribut `classes` les classes rencontrées lors de son entraînement.

**pipeline** Ce package contient notre détecteur de plagiat (classe `PlagiarismDetector`) qui met en place la chaîne de traitement du modèle de détection, du texte brut au résultat plagiat/pas plagiat. Ce détecteur est constitué d'un tokenizer, d'un featurizer et d'un classifier. Ces sont ses attributs. Il dispose de la méthode `train` pour s'entraîner sur des textes (cela consistera en l'entraînement, `fit`, de son featurizer et de sont classifier) et de la méthode `classify` pour déterminer si un texte est plagié ou pas.

### 4.1.2 experiments

Les expérimentations réalisées sur notre modèles et les outils développés pour la cause, notamment le manager d'experimentations (`ExperimentManager`), l'évaluateur des métriques (`Metrics`) se trouvent dans ce package. `Metrics` dispose de méthodes statiques permettant d'évaluer l'accuracy, la précision, le rappel, la balanced accuracy et le score f1 d'un modèle. `ExperimentsManager` permettra lui de lancer les expériences sur les modèles évalués et les données qui lui ont été fournis. Le fichier `experiments.py` contient les expérimentations à proprement parler.

### 4.1.3 gui

C'est le package dédié à la construction de notre interface graphique. Elle est conçue selon l'architecture MVC.

### 4.1.4 util

Nous avons nécessiter des utilitaires qu'il a fallu nous même définir. Elles sont contenues dans le fichier `util.py`, seul fichier du package. Il s'agit entre autres de lecture de fichier ou de dossier (`read_file` et `read_files`) et de la partition d'ensemble (`partition`).

### 4.1.5 tests

Des tests unitaires ont été réalisés sur les éléments non triviaux de notre modèle, notamment la tokenization et la featurization.

## 4.2 Chaîne de traitement

La description du paquetage ci-dessus dessine déjà la chaîne de traitement mise en place. Pour la classification de textes, lorsqu'un détecteur reçoit le texte à classifier, il est transformer en tokens (le featurizer assure en réalité cette étape) que le featurizer utilisera pour extraire les caractéristiques qui seront transmises au classifier pour la prédiction. Pour ce qui est des expérimentations, la chaîne de traitement commence par le choix des modèles à évaluer qui sont stockés par le manager d'expérimenattions. C'est ce dernier qui assure la pipeline du modèle de détections puis donne les performances des modèles qui pourront être visualisées.

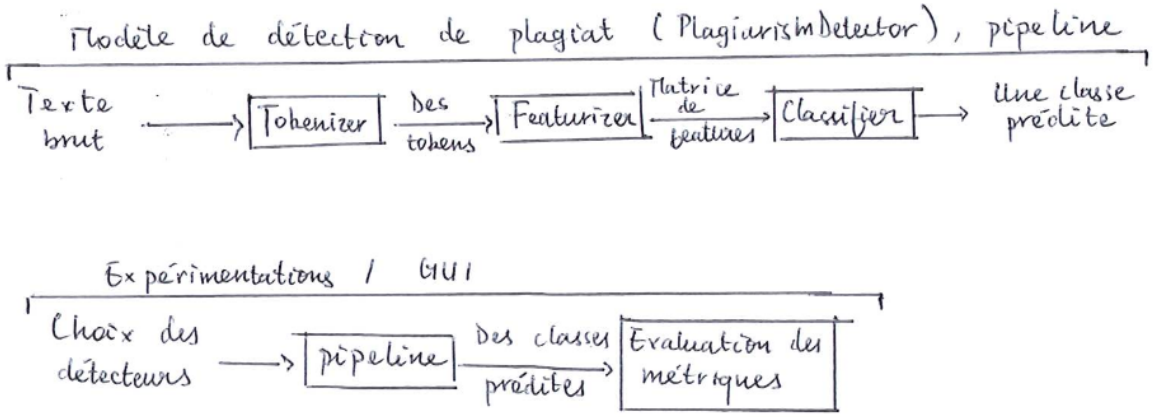



FIGURE 6 – Pipelines

# 5 Experimentations

## 5.1 Pré-analyse du corpus de données utilsé

Avant de se lancer dans les différentes implémentations et évaluations de nos modèles, il a été nécessaire d'analyser le corpus de données qui nous a été fourni. De cela se sont dégagés la structure du corpus, les types de textes inclus, les différents degrés de plagiats impliqués ...

### 5.1.1 Description

Le corpus, en anglais, a été créé expressement pour des études sur la détection de plagiat par le département Informatique de l'université de Sheffield . Il a été demandé à des étudiants de rédiger des réponses courtes à des questions liées à l'informatique qu'étaient les suivantes : "What is inheritance in object oriented programming ?", "Explain the PageRank algorithm that is used by the Google search engine.", "Explain the Vector Space Model that is used for Information Retrieval.", "Explain Bayes Theorem from probability theory." et "What is dynamic programming ?". Pour simuler du plagiat, certains étudiants avaient été orientés vers la source Wikipédia traitant du sujet qui leur avait été confié. Et pour simuler le non plagiat, il a été tout simplement demandé à d'autres d'utiliser leurs propres connaissances. Les différents degrés de plagiat suivant ont été donc définis :

— **Near copy** : demande de copier tout simplement de la source

— **Light revision** : demande de baser la réponse sur la source, avec autorisation de modifier légèrement sa structure
— **Heavy revision** : demande de paraphraser la source, d'utiliser ses propres structures
— **Non-plagiarism** : demande d'utiliser ses propres connaissances pour répondre à la question (possibilité d'utiliser d'autres ressources mais autres que la source Wikipédia)
— **Original** : les textes sources

### 5.1.2 Tailles du corpus

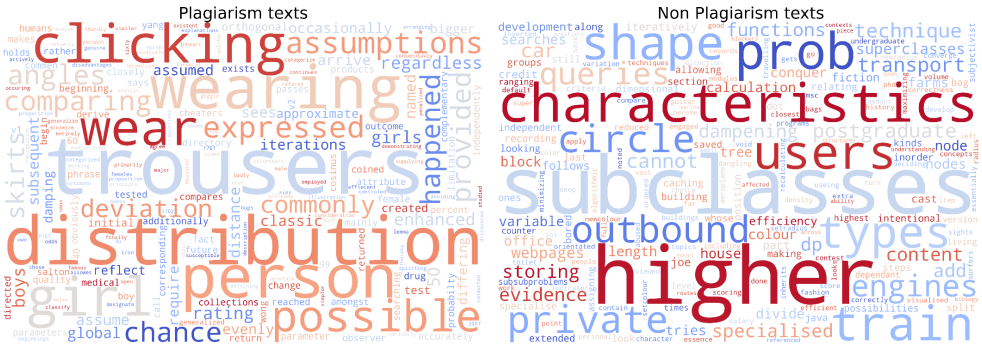
Voici le profil du corpus, décrit par l'université :

> "Le corpus contient 100 documents (95 réponses fournies par les 19 participants et les cinq articles sources issus de Wikipédia). Les textes des réponses contiennent un total de 19 559 mots (22 230 tokens uniques). Les pages Wikipédia totalisent 14 242 mots après conversion en texte brut et suppression des références aux URL. La longueur moyenne des fichiers dans le corpus est de 208 mots (écart-type de 64,91) et de 113 tokens uniques (écart-type de 30,11). 59 des textes (62 %) ont été rédigés par des anglophones natifs. La langue maternelle variait pour les auteurs des autres textes. Pour chaque tâche d'apprentissage, il y a 19 exemples de révision lourde, 19 exemples de révision légère, 19 exemples de copie proche et 38 exemples non plagiés, écrits indépendamment de la source Wikipédia."

### 5.1.3 Qu'avons-nous analysé ?

Après lectures des textes produits par les étudiants, l'aspect le plus visible (pour l'esprit humain) du corpus est que pour un sujet donné, les textes se ressemblent tous, de l'original à la copie. Ce qui est tout de même logique. Par contre, pour les textes plagiés, les paraphrases de l'original ont été facilement identifiables ; mais encore une fois, on le voit parce qu'on est humain. Pourrait-on faire voir à l'ordinateur de pareilles paraphrases ? Il a fallu voir quels autres caractéristiques pouvaient être discrimantes à part ces aspects intuitifs.

Voir quels mots revenaient le plus dans les textes plagiés, et lesquels étaient plus présents dans les textes non plagiés nous a semblé le meilleur choix. Ces mots étaient aux premiers abords les mêmes pour les deux classes, bien évidemment, vu que les textes traitaient tous des mêmes sujets. Ils ont donc été jugés non discriminants et retirés des résultats. Les résultats exploitables obtenus ont été représentés sous forme de nuages de mots (wordclouds) que voici :




(a) Wordcloud des textes plagiés (b) Wordcloud des textes non plagiés

FIGURE 7 – Wordclouds

Aussi, il a fallu déterminer la meilleure frontière entre plagiat et pas plagiat à partir des cinq classes mentionnées ci-dessus. L'intuition considérera une "heavy revision" comme du non plagiat, mais la pré-analyse nous a fait remarquer des textes comme celui du *g2pC-taskd* à propos du théorème de Naive Bayes, qui est une "heavy revision" de l'original *orig-taskd*, mais qui a une structure trop identique par rapport à son original. Nous avons donc considéré comme non plagiat seuls les textes originaux et ceux marqués "non plagiarism".

## 5.2 Évaluations de nos modèles

### 5.2.1 Métriques évaluées

Il a fallu évaluer plus d'une métrique, une métrique seule ne pouvant nous décrire qu'un seul aspect de nos modèles. Cela a inclut l'étude des vrais positifs (VP), des faux positifs (FP), des vrais négatifs (VN) et des faux négatifs (FN), les positifs étant les plagiats et les négatifs les non plagiats.

**Accuracy**

$$\text{accuracy} = \frac{\text{nombre_prédictions_correctes}}{\text{nombre_de_prédictions_totales}}$$

C'est la métrique la plus intuitive. Elle mesure la proportion de labels correctement prédits par notre modèle.

**Balanced Accuracy**

$$\text{recall_pos} = \frac{TP}{TP + FN} \tag{18}$$

$$\text{recall_neg} = \frac{TN}{TN + FP} \tag{19}$$

$$\text{Balanced Recall} = \frac{\text{recall_pos} + \text{recall_neg}}{2} \tag{20}$$

L'accuracy ci-dessus peut être parfois très trompeuse, notamment lorsque les classes sont déséquilibrées. Par exemple, évalué sur un corpus de test contenant 90 textes plagiés et 10 textes non plagiés, un modèle naïf qui labellise tous les textes comme plagiés aura une accuracy de 90%. Le rôle de la balanced accuracy est de corriger cet effet de déséquilibre en faisant la moyenne des taux de détection des deux classes.

**Précision**

$$\text{precision} = \frac{\text{VP}}{\text{VP} + \text{FP}}$$

La précision est la fiabilité du modèle sur ses prédictions positives. Autrement dit, sa capacité à ne pas labelliser comme plagié un texte qui ne l'est pas.

**Recall**

$$\text{recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

Le recall (rappel) mesure la proportion de cas de plagiat correctement détectés par le modèle. C'est la capacité du modèle à détecter tous les plagiats.

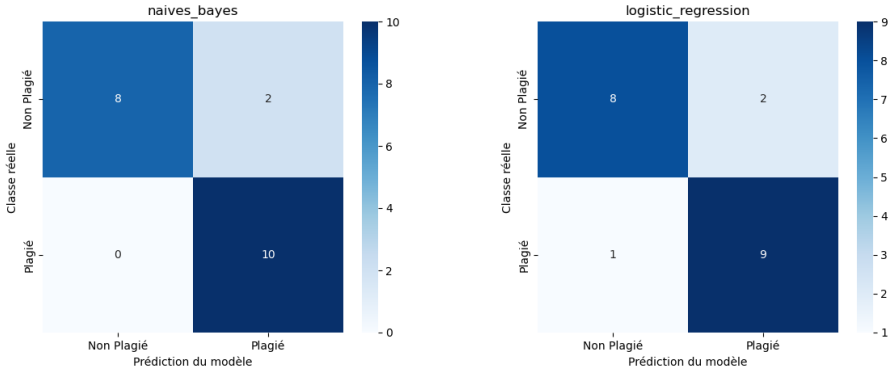
**F1-score**

$$\text{F1-score} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Il fait la moyenne harmonique de la précision et du rappel. Formellement, il mesure la capacité de notre modèle à détecter tous les plagiats sans pour autant accuser à tort les textes qui ne sont pas plagiés.

Les VP, FP, VN et FN pourront par ailleurs être visualisés à l'aide d'une matrice de confusion, matrice de la forme :

$$\begin{bmatrix} \text{VN} & \text{FP} \\ \text{FN} & \text{VP} \end{bmatrix}$$




(a) Naive Bayes (b) Logistic Regression

FIGURE 8 – Matrices de confusions après train sur 80 textes

### 5.2.2 Quels modèles sont les plus performants pour détecter du plagiat ?

Pour assurer la crédibilité des études réalisées présentées ci-dessous, les résultats présentés ci-dessous pour les évaluations de métriques sont les moyennes obtenues en lançant les expérimentations 1000 fois sur différentes partitions de nos données. Cela a nécessité de gros temps de calculs.

**Classifiers** Occupant une place centrale dans le processus de détection, les classifiers ont été notre premier objet d'expérimentations. Les résultats obtenus classent les 6 classifiers que nous avons implémentés en 3 catégories. La première catégorie, les meilleurs, aux performances pouvant dépasser les 85%. Ce sont là le Naive Bayes qui prouve son efficacité malgré sa forte hypothèse d'indépendance des features, et l'AdaBoost qui montre l'importance de l'attribution de grands poids aux modèles plus performants. Le BaggingClassifier et la Logistic Regression sont aussi performants (environ 75%), mais d'une différence non négligeable par rapport aux premiers et sont donc de la deuxième catégorie. La troisième regroupe donc le Perceptron et les Décision Trees, qui sont le moins adaptés pour notre ensemble de données (performances jugées non intéressantes d'environ 60%, et parfois moins).

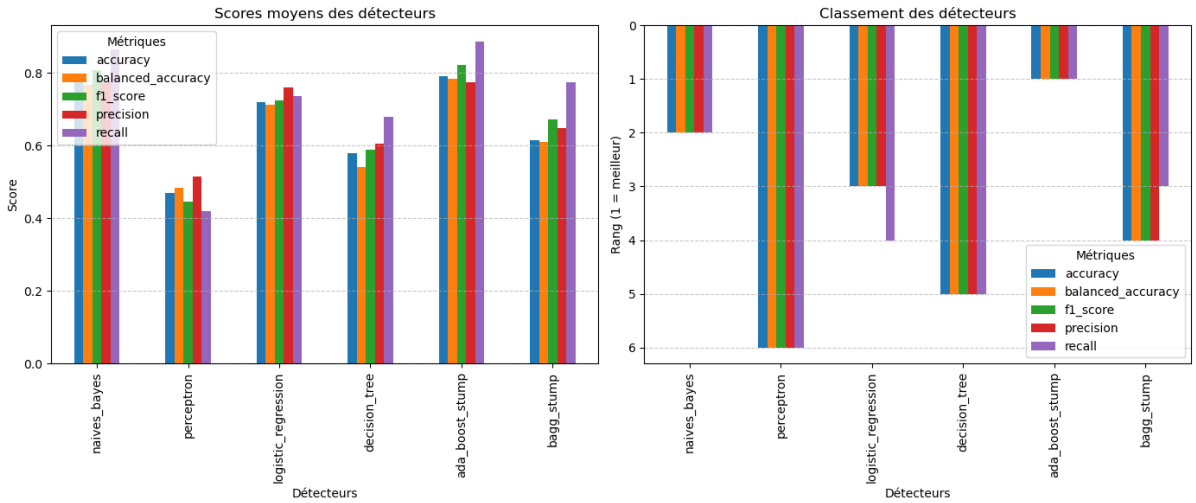



FIGURE 9 – Comparaisons des classifiers

**Featurizers** Le choix du featurizer n'est pas non plus à négliger. Là encore, c'est la simplicité et l'intuition, notamment des caractéristiques extraites par le CountVectorFeaturizer, qui l'emporte, à la quasi-unanimité des classifiers, mais légèrement.

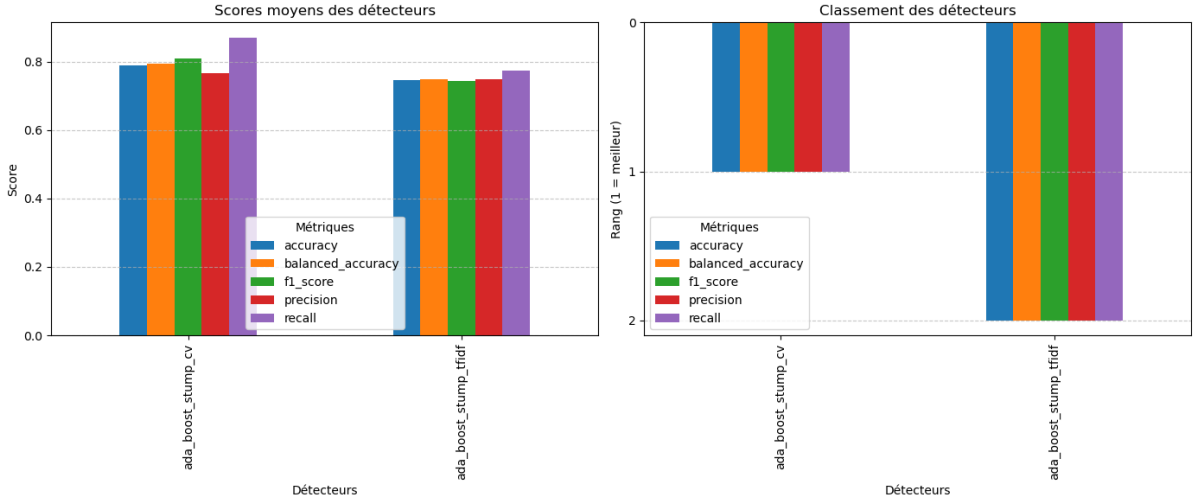



FIGURE 10 – Comparaison de Count Vector et TFIDF avec AdaBoost

Par ailleurs, pourrait-on au delà du choix des featurizers, rendre les featurizers eux-mêmes plus performants en les faisant travailler sur des sacs de mots de tailles variées et plus grandes ? Pour cela nous avons testé nos featurizers avec différents ngrams. L'extraction de caractéristiques sur des mots seuls (ngram [1,1]) s'est révélée être la moins efficace sans pour autant être significativement plus mauvaise.

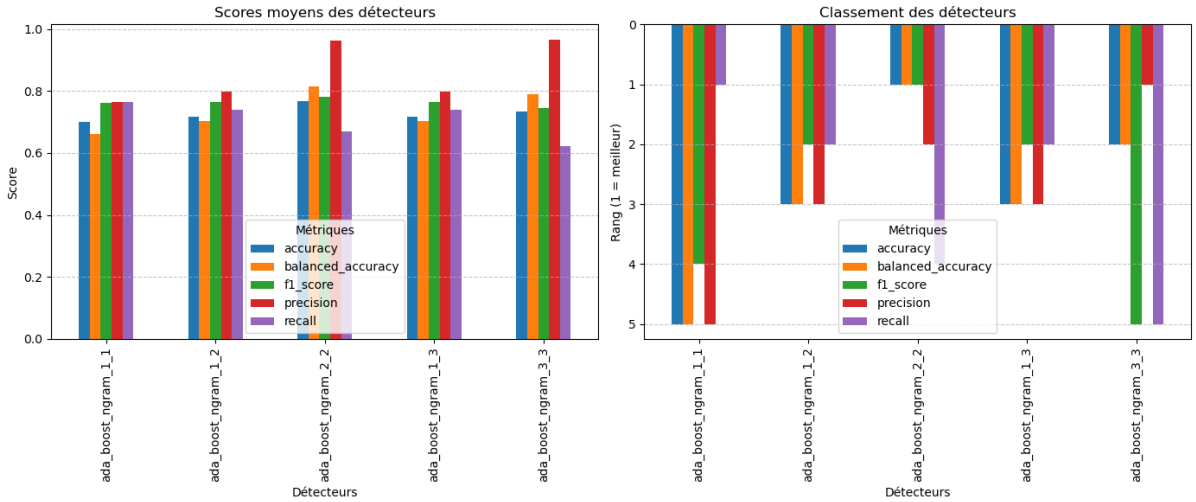
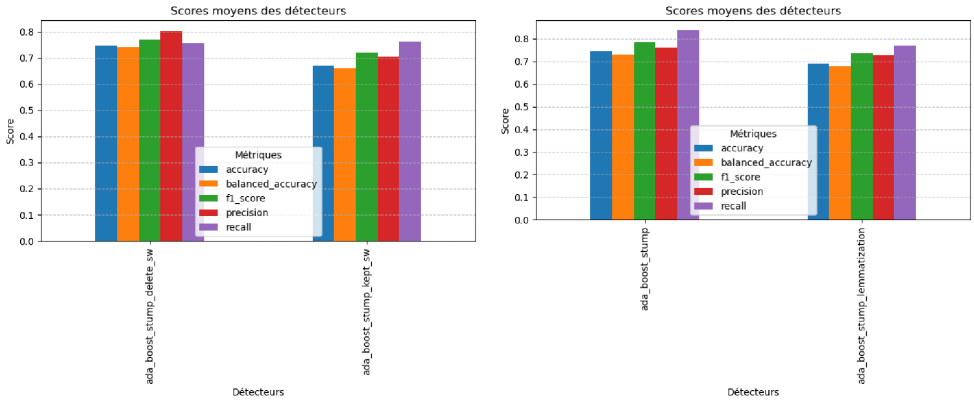



FIGURE 11 – Comparaisons sur les ngrams avec CountVectorizer et Adaboost

**Impact des préprocesseurs** Les stop words sont des mots présents très fréquemment dans le langage. Ils devraient donc être moins discriminants pour différencier différentes classes de textes. Est-ce vraiment le cas ? Pour notre part, enlever ou garder les stopwords lors de la tokenization s'est révélé sans impact significatif. Mais cela est sans doute dû à la taille modeste de notre corpus.

Qu'en est-il de la lemmatization ? Au lieu de considérer les mots, considérons leur forme canonique, qui est plus discriminante, surtout dans le cas de la paraphrase. Mais là aussi, notre corpus n'a pas permis de dégager son éventuel impact.




(a) Suppression de stopwords (b) Lemmatization

FIGURE 12 – Impact du préprocessing

**Impact de l'apprentissage** Plus un modèle apprend, plus il est performant. C'est une intuition ; et elle a été validée par nos expérimentations. D'éventuels pièges de sur-

apprentissage n'ont pas pu être détectés dans notre cas. Les modèles atteignent généralement leur pic de performance après entraînement sur 80 textes (limite fixée pour avoir un corpus de test significatif vu que nous avons 100 textes au total).

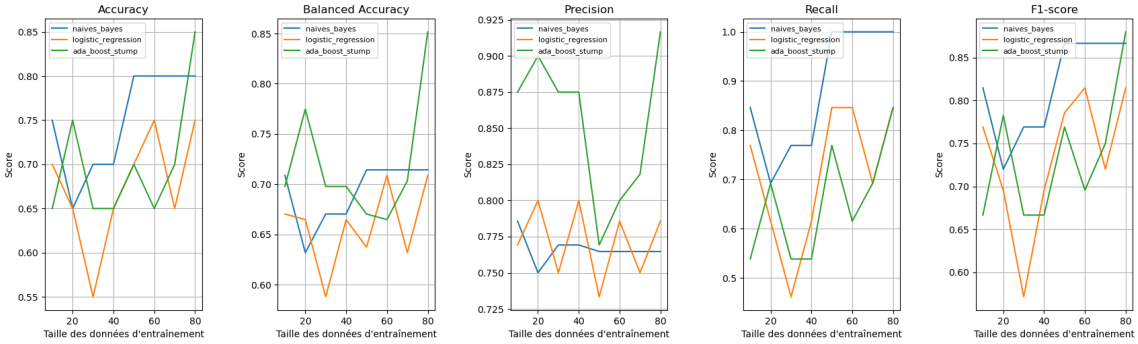
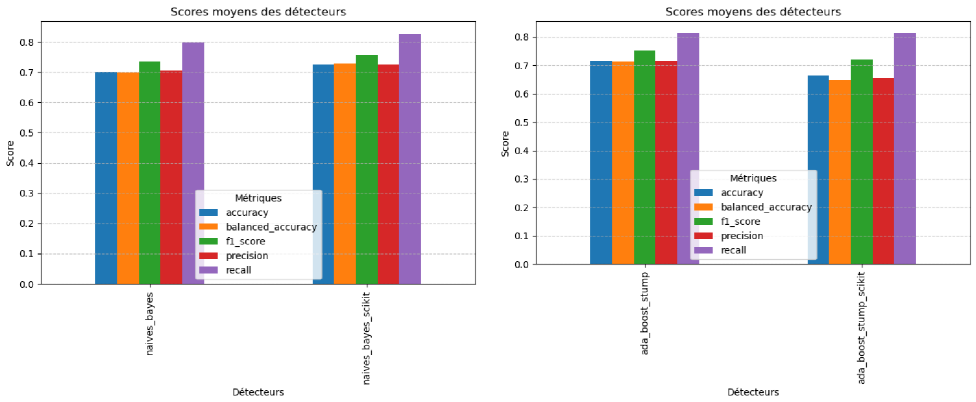



FIGURE 13 – Mesures de l'impact de l'apprentissage

**Comparaisons avec Scikit** Nous avons également comparé nos modèles à ceux du module de scikit que nous avons pris comme référence. Cela nous a permis de confirmer la robustesse des modèles que nous avons nous-mêmes développés. Car en effet, nos performances sont très proches de celles de scikit.




(a) Naive Bayes (b) AdaBoost

FIGURE 14 – Comparaisons avec les modèles de scikit

## 5.3 Interface graphique

Pour réaliser des expérimentations plus facilement, nous avons développé une interface graphique simple à prendre en main. Elle nous permet d'évaluer toutes les métriques mentionnées ci-dessus et de comparer les modèles entre eux. Son point fort est que nous pouvons choisir tous les hyper-paramètres des modèles que nous voulons tester. Augmenter la profondeur des arbres de décisions est-il une bonne idée ? Quel est l'impact du learning rate pour un modèle comme le Perceptron ou la logistic regression ? Les réponses à ces questions pourront être visualisées sur notre interface graphique.

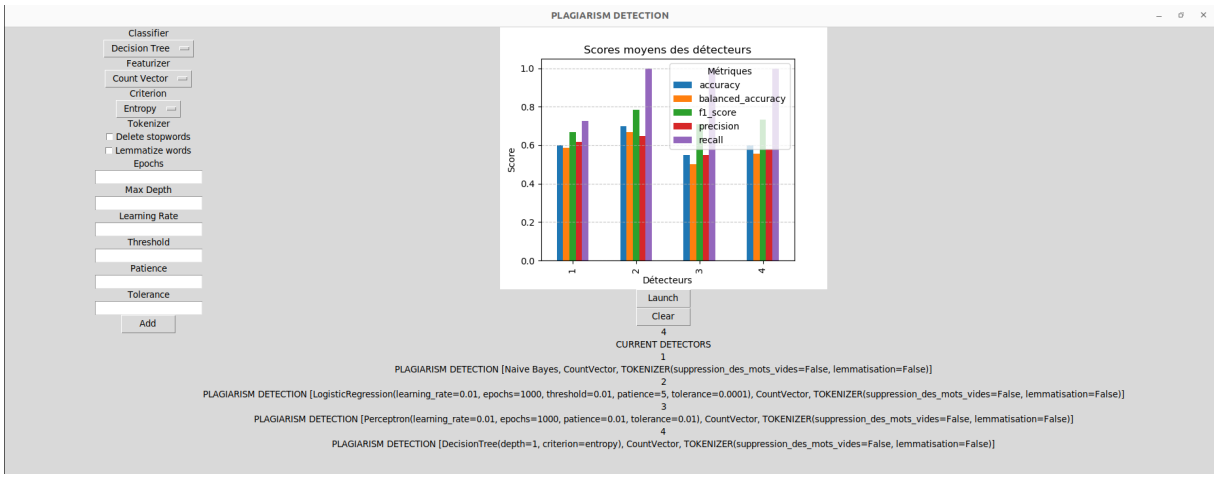



FIGURE 15 – Notre GUI

## 5.4 Limites

La qualité d'un modèle de détection de plagiat dépend fortement de la qualité du corpus sur lequel il a été entraîné. À celui que nous avons utilisé, nous pouvons faire plus d'une remarque. D'abord, il est de taille assez modeste (seulement 100 textes), et les textes sont aussi de taille modeste vu que sa conception est la réponse à des "short answer question". Ensuite, il est assez spécifique. Il traite de sujets spécifiques, mais en plus liés uniquement aux sciences de l'informatique. Puis, les participants à sa création sont au nombre de 19 et tous des universitaires. Par ailleurs, il est assez difficile de dire en réalité ce que c'est qu'un texte plagié. Prenons l'exemple du near copy, le plus haut degré de plagiat. Considérons qu'un individu B recopie la production d'un individu A. Comment sans ce contexte pourrait-on dire, en analysant les caractéristiques de ces textes, de surcroît en le faisant faire à un ordinateur, que le texte de B est intrinsèquement plagié et que celui de A non ? Vu que le texte A a été produit par un esprit humain, B l'est aussi ; la copie est l'originale si on ne la sait pas être copie.

# 6 Conclusion

Dans ce travail, nous avons exploré plusieurs modèles de classification pour la détection de plagiat, en nous concentrant particulièrement sur l'évaluation de leur performance à travers des métriques adaptées telles que la précision, le rappel, et la balanced accuracy. Nous avons utilisé des techniques d'agrégation des classificateurs comme le *bagging* et l'*AdaBoost* pour améliorer la robustesse des modèles et leur capacité à généraliser sur des données non vues.

L'analyse des résultats a permis de mettre en lumière l'importance de l'apprentissage dans les performences du modèle. Nous avons aussi pu dégager les modèles plus performants et tester l'impact d'aspects comme le pré-traitement des textes ou la taille des sacs de mots.

De plus, la pré-analyse, en utilisant des outils tels que les nuages de mots, a permis d'identifier les termes récurrents et discriminants, fournissant ainsi un aperçu précieux sur les différences lexicales entre les textes plagiés et non plagiés.

Enfin, des pistes d'amélioration existent l'enrichissement des jeux de données, pour rendre le modèle plus robuste et capable de détecter des cas de plagiat plus subtils.